

VHDL simulation environments for AOC2 course projects

DIIS - EINA - Univ. Zaragoza

February 18, 2025

The projects can be run on any VHDL simulator. In this document we have included information on how to install some of them. This information is subject to change as there have been regulatory changes in the United States of America that affect free ModelSim licenses.

1 GHDL & GTKWave

GHDL is an open source toolchain for analyzing, compiling assembling and simulating VHDL designs. It is available for Linux, MacOS, and MS Windows. It is managed through the command shell (in the case of MS Windows, Cygwin, CMD or PowerShell). You can download the sources, keep them updated with `git` and compile the whole toolchain, or install it as a package on either Debian/Ubuntu or MacOS (see Sec. 1.1).

GHDL packages a code generator for MacOS/x86 by default (`mcode`). For Linux you have to use `llvm` or `gcc`. In these instructions we assume Linux/x86 and install the `ghdl-llvm` variant.

GHDL is not an integrated environment and requires a separate viewer to display the waveforms. GTKWave, like GHDL, is easily installed as a package on any Linux, MacOS or Windows. For Mac users, GHDL+GTKWave is the only way we know of to simulate VHDL.

1.1 Installation

1.1.1 Hendrix account

On the lab PC, log in to CentOS with your hendrix account. On the password screen you can choose the graphical environment (`xdm` recommended; it is way faster).

Edit the file `.software` and add the following line:

```
1 ghdl
```

Then, run for example in a text terminal:

```
1 ghdl --version
```

Gtkwave can be run from the application menu of the graphical environment, or in the background from the command line:

```
1 gtkwave &
```

1.1.2 Debian and Ubuntu Linux

On Debian style distros, run:

```
1 sudo apt-get update
2 sudo apt-get install ghdl-llvm
3 sudo apt-get install gtkwave
4 ghdl-llvm --disp-config
```

This last step simply checks the installation. Note that the binaries `ghdl` and `ghdl-llvm` have been created in `/usr/bin`, and that `ghdl-llvm` is a driver to the second one. Defining `ghdl` alias `ghdl="ghdl-llvm"` would have the same effect.

The `apt` manager currently installs `ghdl 1.0`. In Ubuntu¹, you can use the snap manager to install any version (the latest by default):

```
1 sudo snap install ghdl
2 ghdl --version
```

1.1.3 Windows and MacOS

We have not found any reliable `ghdl` package directly installable on Windows. A Linux virtual machine is an option, but GTKWave will likely give problems. As of this writing, `ghdl` on WSL Linux distros presents a number of issues, and requires switching to Windows to run GTKWave. Our recommendation is to install [MSYS2](#) ([installer here](#)), because the `ghdl` project provides support for this platform. Open a MSYS2 console² and run in that terminal:

```
1 pacman -S mingw64/mingw-w64-x86_64-ghdl-llvm
2 pacman -S mingw-w64-x86_64-gtkwave
```

Get updates running `pacman -Syu`

Click the MSYS2MINGW64 terminal, or open it running `mingw64` from PowerShell or:

```
1 C:\msys64\mingw64.exe
```

¹There are binaries for GHDL over Ubuntu 24.04 at the GHDL website, but issues have been reported with the LLVM libraries

²From PowerShell just run `msys`

That terminal is highly configurable, and is ready for running the `ghdl` commands. You will have to `cd` to your Windows working folder, e.g.:

```
1 cd /c/Users/luke/mystuff
```

A smarter way is to create a link (named *junctions* in MsWindows) within your MINGW64 home directory, which for a default MSYS installation is at (say you are user `luke`):

```
1 C:\msys64\home\luke
```

Right-click on the icon of the Windows folder whose path you want to link to your MINGW64 home, and copy the absolute Windows path to the folder, like e.g.

```
1 "C:\Users\luke\mystuff"
```

Then, you could create a symbolic link just from your MINGW64 terminal, leveraging `ln -s` but you might have problems with Windows2Unix path conversions and funny characters. Long story short, way safer, open a `cmd.exe` console in your MINGW64 home (see above) and run Window's `mklink`³:

```
1 mklink /J "C:\msys64\home\luke\<linkname>" "C:\Users\luke\mystuff"
```

This way when you open up a new MINGW64 console, you just `cd` to the `linkname` and you're done.

1.1.4 VSCodium

VSCodium is a free version of Microsoft VSCode. You can easily turn it into a friendly and productive integrated VHDL environment on (native) Linux or Windows⁴. The editor supports regex find and replace, symbol automated completion, folding, and keeps block headers (entity, procedures...) static while scrolling inside the block. Follow this steps:

- Get the proper installer for your system from <https://github.com/VSCodium> and install VSCodium.
- Open VSCodium, open the extension manager and install the first extension for VHDL
- Go to the command palette (`Ctrl-Shift-P` in Windows) and in the Text Editor settings → Files, check *Auto Guess Encoding*
- Open the folder where you have your project or lab code. You'll see all the files in a panel and you'll be able to edit and navigate around easily.

³Mind that the link name and target name are reversed w.r.t the `ln` Unix command

⁴We haven't tried it out on Mac yet. Make us know if you do it)

- You can open integrated terminals, but running `mingw64` will open up an external MingW64 terminal, from which you can run the common `ghdl` and `GTKwave` workflow.
- Git and the GitHub interface are integrated into VSCodium: you can turn your project's folder into a git repo and then upload it to GitHub, or clone a GitHub repo right from there.

1.2 Basic workflow

There are many ways to proceed with `ghdl` but we strongly recommend the following basic *workflow*. We start from the directory where you have your vhd sources. We assume the extension `.vhd` for the files, that the VHDL ENTITY with the testbench is named `testbench`, and that in that directory you have created a subdirectory for the project which we will call here `WORK`, to avoid cluttering the directory where you keep your vhd sources.

```
1 ghdl -i --ieee=synopsys -fexplicit --workdir=WORK *.vhd
2 ghdl --gen-makefile --ieee=synopsys --workdir=WORK testbench > Makefile
3 ghdl -m --ieee=synopsys -fexplicit --workdir=WORK testbench
```

In step 1 you are creating the project in the directory `WORK`. In step 2 we create a file `Makefile` to build an executable file in step 3 with `ghdl -m`⁵

Now you can run

```
1 ./testbench --stop-time=500ns --wave=test.ghw
2 gtkwave test.ghw &
3 ghdl --clean --workdir=WORK
```

In step 1 above, we run the executable; set the time to whatever time you need to fully simulate the testbench or the test program in case. The last step clears everything, if your changes are getting muddied. Run `rm -r WORK` if you reached your panic point, and rebuild the project.

The `gtkw` files contain waveform configurations with the signals you want to observe. You can create your own and save / load them in `GTKwave` from File → Save File .

Adding new source files to the project is easy with `-i`, but you will have to rebuild `Makefile`. Removing source files means either modifying `Makefile` by hand, or deleting the source and rebuilding `Makefile` from the scratch. Keep a copy of the RAM files outside the directory where you build the project, or have multiple RAMs (VHDL vectors) inside each RAM file (e.g. `RAM-D.vhd`, `RAM-I.vhd`), commenting out all of them but the one you want to use.

⁵Depending on your `ghdl` version, `gen-makefile` does not pass the option `-fexplicit` to the `Makefile`, and therefore it is necessary to edit this `Makefile` and add this option to the variable `(GHDLFLAGS= --ieee=synopsys -fexplicit --workdir=WORK`. Keep this in mind when installing `ghdl` on your system.

1.2.1 Cleaning up the project

```
1 ghdl --clean --workdir=<your workdir>
2 ghdl --remove --workdir=<your workdir>
```

Command 1 removes the executable and all the object files. If you want to rebuild the design at this point, just do the make command (`ghdl -m ...`) as shown above.

Command 2 removes the executable, all the object files and the library file. If you want to rebuild the design, you have to import the sources again and make the design.

1.3 Helpful resources

1. [ghdl on GitHub](#)
2. [GTKWave on GitHub](#) GTKWave landing page recommended for Mac users.
3. [Our quick tutorial on Moodle \(YouTube\)](#)
4. `\man ghdl` (in Linux). The best `ghdl` manual you will find.

2 ModelSim Microchip

- **Register** by accessing [this link](#). Use your unizar email address.
- Download the environment. To do this select the Software tab in the link above (Fig. 1), locate the version in Fig. 2 and download it.
- Unzip the .zip; nothing needs to be installed. The executable is in `modelsim\modeltech\win32acoem`.
- **Get the lincese** in [this link](#).
- Click on *Register a free license* (Fig. 3) and download the license (Fig. 4). If you find different license types to choose from, the appropriate one for Windows is: *Liberio Evaluation Node Locked License for Windows*. Upon request, you will be asked for the serial number of your hard disk C:. They explain how to get that serial number right there, running `Vol C:` in a command window. This is just to obtain a license that will only work on the computer with that specific hard disk. You will get the license by email in a few minutes.
- **Copy the license** (`license.dat`) to the directory `\C:flexlm`.
- Verify that the variable `LM_LICENSE_FILE` is defined as an environment variable, and add the address of the license: `c:\flexlm\license.dat`.

Figure 1: Page to download ModelSim

ModelSim ME v10.5c Stand-Alone Software Release (08/03/2017)

ModelSim ME v10.5c for Windows®

Figure 2: Page for ModelSim download

3 ModelSim - IntelFPGA

Intel also allows you to download ModelSim from [this link](#).

- Select *Intel Quartus Prime Pro Edition* (e.g. the 21.2)
- Select download *Individual Files* and then *ModelSim-Intel*FPGA*, files:
 - ModelSim - Intel FPGA Edition (includes Starter Edition)
 - ModelSim - Intel FPGA Edition (includes Starter Edition) Part 2
- Install the file as administrator. On Windows, run the file `ModelSimProSetup-21.2.0.72-windows.exe` for version 21.2.0.72-
- A window will appear where you must select “ModelSim Starter Edition”, which does not require a license file.
- Accept the license terms and select the installation directory.

4 Active-HDL Student Edition

The Aldec company has a very powerful Windows simulator for VHDL, among others. The environment is similar to Modelsim. It can be downloaded from [this link](#). You will be asked for registration (you have to do it with your unizar email). you'll get an email with a link for downloading.

5 Xilinx ISE

Xilinx ISE. The Xilinx ISE Webpack environment is available for Windows and Linux. It requires registration with Xilinx. On Windows 10 it runs on a Linux virtual machine totalling about 38 GB:

License Selection

Libero Paid and DirectCore Licenses – Can be purchased through our online purchasing portal

Paid licenses are available as NodeLocked and Floating based. Renewal Licenses are discounted per seat. Incremental licenses can be added to floating licenses during the year for pro-rated pricing.

We offer multiple licenses to design with our FPGA and SoC design tools. Most of the software tools and FPGA IP cores are freely available but a few high-value IP cores and resources needed to work with high-density FPGAs required paid licenses.



Figure 3: License registration

Seats	Registration Date	Caducidad	Download License
1	19-Jan-2022	20-Mar-2022	Download License

Figure 4: License download

- It is advisable to install Oracle MV VirtualBox first.
- If it already exists, the Xilinx installer recognizes it, asks if you want to use that version and leaves three links on the Windows 10 desktop, of which you will only use the *Project Navigator*.
- If you already have a virtualizer such as Oracle VM Virtual Box on your Windows it will recognize it and give you the option to use it. If you do not have it, install the virtualization software as well.
- Clicking on *Project Navigator* starts the virtual machine, which launches a Linux with its included desktop, and then automatically launches the ISE as well.

We have detected a few cases on Windows 10 Home Edition where the above procedure has given problems. In that case, you can first create a VM with VirtualBox, starting from any Linux distribution (Ubuntu or whatever you want), and then download and install the Linux version of ISE on that VM.

However, installing it on Windows as a separate machine has two advantages:

1. You can start the whole chain from the link to the Windows desktop.
2. The creation of a shared folder between the virtualized Linux VM and the host Windows is a must. This step is integrated into the ISE for Windows 10 installation process.

5.1 Installing the ISE Webpack

1. Register with XILINX, best with university mail, by logging in:
 - <https://www.xilinx.com/products/design-tools/ise-design-suite/ise-webpack.html>
2. Choose the option *Download ISE WebPACK software for Windows and Linux*.
3. Selet version 14.7
4. Select the installer (there are for Windows and Linux)
5. Download it and unzip it.
6. In the unzipped folder you will find the **xsetup** that installs the ISE:
 - (a) You will be asked to accept all the licenses.
 - (b) Then you have to choose the ISE version. Choose *ISE WebPACK*. The rest are all paid versions.
 - (c) In “installation options” you can deactivate the three options (Acquire a License Key, Install WinPCap, Install Cable Drivers).
 - (d) Choose the installation directory. It asks for about 17GB of space. On our computer it takes about 11 or 12 GB. The installation will take a few minutes.

6 Other environments

Xilinx offers the Vivado environment, which is geared towards working with the brand’s FPGAs. Unlike ISE, Vivado is updated and maintained, and does not require a virtual machine in the case of Windows. The downside is that it is a more complex and bulky environment, with many other tools (even more than ISE) besides the VHDL simulator. The advantage is that it allows you to synthesize your design for a modern FPGA, and see the result in area, cycle time etc.

Finally, to run short simulations you can use the online tool [edaplayground](#). The advantage of this tool is that you don’t have to install anything. Cons are that it is not suitable for large projects (such as AOC2 course projects) and the error messages are anything but clear.